

Министерство науки и высшего образования РФ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Белгородский государственный технологический университет им. В.Г. Шухова»



Кафедра программного обеспечения вычислительной техники и автоматизированных систем
Направление подготовки 09.03.04 Программная инженерия

Выпускная квалификационная работа на тему:

Разработка сервиса хранения аудиоданных с применением алгоритмов упаковки и потоковой распаковки

Выполнил:

Пахомов Владислав Андреевич, студент группы ПВ-223

Руководитель:

ст. пр. Картамышев Сергей Владимирович

Цель работы

Разработка сервиса, позволяющего управлять процессами разработки музыкальных композиций: хранить, версионировать и распространять аудиоданные, в том числе с применением собственного алгоритма упаковки и потоковой распаковки.

Задачи:

- 1 Разработка архитектуры и программного обеспечения для хранения аудиоданных.
- 2 Тестирование разработанного программного обеспечения для хранения аудиоданных.
- 3 Создание руководства пользователя разработанного программного обеспечения для хранения аудиоданных.

Функциональные требования к сервису BackTrack:

- Аутентификация и регистрация пользователей
- CRUD-операции над авторами и группами
- Создание и версионирование композиций
 - Запрет на удаление; только новый релиз
 - Система тегов для версий
 - Чат внутри каждой композиции
- Умные плейлисты с фильтрацией по тегу/версии
- Хранение файлов с проприетарным кодеком
- Поточковая распаковка аудиоданных (chunked)
- Встроенный аудиоплеер с управлением очередью

Проблемы рынка:

- Стриминг использует треть россиян, аудитория растёт
- Продюсеры хранят проекты в локальных папках
- Утеря носителя — потеря всех наработок
- Нет инструментов версионирования и совместной работы
- Готовые сервисы узкоспециализированы

Что даёт разрабатываемый сервис:

- Централизованное облачное хранилище
- Версионирование без права удаления
- Теги (*Demo, Live, Release*)
- Умные плейлисты для разных заказчиков
- Self-hosted: конфиденциальность данных

Сравнительный анализ существующих аналогов

Критерий	Git	Splice	DropBox	BackTrack
Версионирование аудио	+	±	–	+
Облачное хранение	±	+	+	+
Система тегов	+	–	–	+
Плейлисты	–	±	–	+
Чат / комментарии	+	–	–	+
Self-hosted	+	–	–	+
Удобство для продюсера	–	+	±	+

Git и DropBox решают одну задачу; Splice перешёл на маркетплейс сэмплов; Boombox.io близок, но не имеет версионирования и собственного кодека.

Backend

-  **FastAPI**

-  PostgreSQL

-  **docker**®

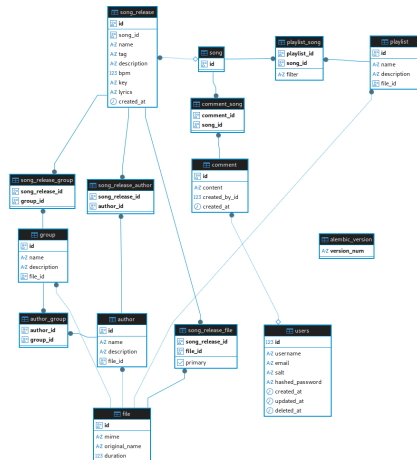
Frontend

-  **React**

-  RxJS

- **VITE**()

Бизнес-сущности (БД):



Аудиоданные:

- Входной формат: WAV (PCM mono)
- Фреймы фиксированной длины: 16 384 амплитуд ($\approx 0,37с$)
- Каждый фрейм — независимый вектор
- После кодирования: сжатый бинарный формат
- При передаче: HTTP Chunked Transfer
- Распаковка: потоковая, без ожидания всего файла

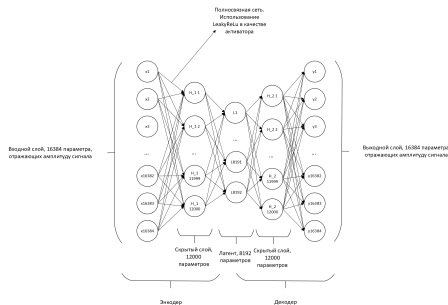
Автоэнкодер (нейросетевой кодек):

- Вход: фрейм $x \in \mathbb{R}^{16384}$ (0,37с, 44,1 кГц, моно)
- Энкодер: $z = f_{\theta}(x)$, $z \in \mathbb{R}^{8192}$ (сжатие $\times 2$)
- Декодер: $\hat{x} = g_{\phi}(z)$
- Функция потерь: MSE + спектральная
- Активатор: LeakyReLU
- 589 824 000 весов; VRAM ≥ 16 ГБ

Гибрид с FLAC:

- FLAC (кодер Хаффмана) + автоэнкодер
- Целевой коэффициент сжатия: $\times 2$

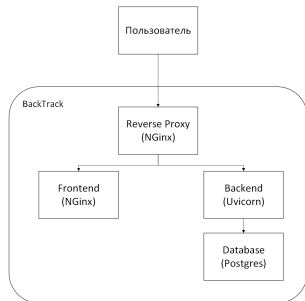
Структура сети:



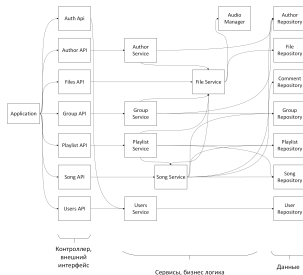
Перцептрон (свёрточные слои):

Локальные фильтры уменьшают число связей — входной слой масштабируется без квадратичного роста параметров.

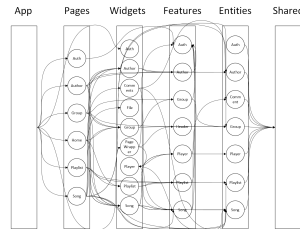
Системная архитектура (монолит):



Архитектура сервера:



Архитектура клиента:



Аппаратура (сервер):

- CUDA GPU ≥ 16 ГБ VRAM
- ОЗУ ≥ 32 ГБ
- Диск ≥ 1 ТБ
- Канал ≥ 10 ГБит/с

Клиент:

- Современный браузер
- Канал ≥ 10 МБит/с

ПО (сервер):

- ОС: Linux (Docker-совместимая)
- Docker + Docker Compose
- NVIDIA Container Toolkit
- CUDA ≥ 13.0
- Python ≥ 3.10
- Node.js (сборка фронтенда)

Восстановление после отказа: ≤ 1 часа (ТЗ А.4.2.в)

Аутентификация и авторизация:

- JWT в заголовке `Authorization`
- Алгоритм подписи: `HS256`
- Payload: `id, username, email`
- Пароль: `bcrypt`-хэш + случайная соль
- Множественные итерации соли (защита от брутфорса)

JWT хранится в RAM клиента — CSRF нерелевантен. Схемы запросов и ответов разделены во избежание утечки внутренних данных.

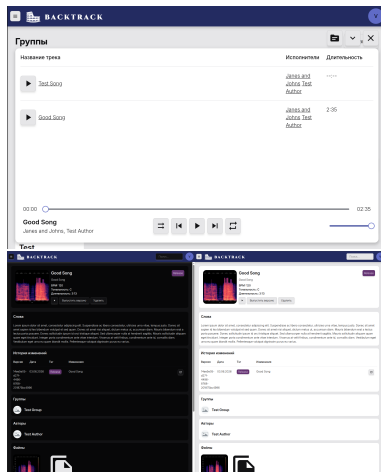
Защита транспорта и клиента:

- Pydantic — валидация всех вх./вых. данных
- Авторизация на уровне STOMP WebSocket
- Файлы вне БД — сокращение атакуемой поверхности
- Self-hosted: данные не покидают инфраструктуру

Формы:

The image shows two screenshots of the BACKTRACK application interface. The top screenshot displays the 'Создать плейлист' (Create Playlist) form, which includes fields for 'Имя' (Name), 'Описание' (Description), 'Песни' (Songs) with a 'Выберите песню из списка' (Select song from list) button, and 'Картинка' (Image) with a 'Выберите файл' (Select file) button and an 'Опять не выбран' (Not selected) message. A 'Создать' (Create) button is at the bottom. The bottom screenshot shows the 'Регистрация' (Registration) form, which includes fields for 'Имя пользователя' (Username), 'Электронная почта' (Email), 'Пароль' (Password), and 'Подтверждение пароля' (Confirm password). It also has a 'Запомнить меня' (Remember me) checkbox and 'Зарегистрироваться' (Register) and 'Войти' (Login) buttons.

Основные экраны:



Обучение нейронных сетей:

- Метрики: SNR и % амплитуд в допуске ($\pm 5/10/20\%$)
- **Лучший результат:** SNR = 17 дБ (разборчивая речь)
- Модель не достигла минимума — возможно дообучение
- Спектральная функция потерь: качество падало с эпохами
- Увеличение нейронов в скрытом слое не оправдало затрат

Smoke-тестирование:

- Сценарии по пунктам A.4.1 ТЗ
- Пройдены: авторизация, CRUD, версионирование, плейлисты, плеер, загрузка файлов
- Healthcheck бэкенда и фронтенда

Потоковая распаковка:

- HTTP Chunked Transfer подтверждён: воспроизведение начинается до загрузки всего файла

В ходе работы разработан и протестирован сервис BackTrack:

- **Исследованы** алгоритмы хранения аудио (FLAC, MP3, AAC, Opus); возможности использования гибридной схемы FLAC + автоэнкодер
- **Разработан** сервер на FastAPI + PyTorch + NVIDIA CUDA
- **Разработан** фронтенд на React + RxJS (Feature Sliced Design)
- **Проведено** Smoke-тестирование всех функциональных требований
- **Составлено** руководство пользователя